Assignment 1:

1. Objective

Develop a Mini Payment API that simulates a payment gateway transaction. The goal is to demonstrate your ability to handle sensitive data, enforce security protocols, and implement conditional business logic

.

2. The Endpoint

POST `/api/v1/pay`

(Minimum) Request Payload:

- `order_number`: merchant order number
- `card_number`: 16-digit string
- `expiry_date`: MM/YY format
- `cvv`: 3 or 4 digit string
- `currency`: ISO 4217 format A3
- `cardholder_name`: String
- `email`: Valid email address
- `amount`: Decimal (e.g., 10.00)

3. Requirements & Deliverables

The API should process the request and return a JSON response including a `status`, `message`, and an `acquirer_reference`.

- Logic Based on the Amount's decimal place:
    - 00 = success
    - Others = rejected

      e.g., If `amount` == 10.00, return Payment Success
      If `amount` == 10.05, return Payment Reject

    - Status will be "APPROVED" for success and "DECLINED" for rejection
    - There ***may*** be other statuses "FAILED", "UNKNOWN"
- Response Fields
    - `transaction_id`: A unique UUID generated by your system.

    - `acquirer_reference`: A unique mock reference number returned by the "bank."
    - `response_code`: response code will follow the logic behind the amount logic. e.g if the `amount` is 10.00, the response code will be 00
    - `status`: As per above
    - `timestamp`: ISO 8601 format.
    - `amount`: decimal
- Data Storage (Bonus)
    - Store the transaction info in a database. Follow the “data security rule mentioned below.

4. Non-Functional Requirements

The solution must address the following architectural concerns:

- Required logging for the requests/responses
- Data Integrity & Validation: * Implement strict schema validation (e.g., ensure the card number is numeric, the email is valid, and the expiry date is in the future).
- Data Security (PII & PCI Compliance): * Do not log/store sensitive data. Ensure CVV and Card Numbers are masked or excluded from application logs.
    - Demonstrate how you would handle sensitive strings in memory (e.g., encryption or secure types).
- Client Identity & Authentication: * The API must be protected. Implement a mechanism to identify the caller (e.g., API Key, Bearer Token, or Basic Auth).
- Idempotency: * (Bonus) How does your API handle a user hitting the "Pay" button twice? Implement or describe an idempotency key strategy.

5. Technical Constraints

- Language: Java or C# preferred.
- Framework: Spring Boot (Java) or .NET Web API (C#).
- Documentation: Provide a README explaining how to run the API and a sample `curl` command or `postman collection` to test the 10.00 vs 10.05 logic.

Assignment 2: Reconciliation

1. Objective

Develop a batch processing utility that compares two financial datasets (**List A** and **List B**) to identify discrepancies.

2. The Data Sources

You are provided with two CSV files (links provided below).

Click Here for Link A

Click Here for List B

3. Requirements & Deliverables

Your program must read List A and List B as input files and generate three (3) output CSV or xlsx files based on a **Unique Reference** comparison (defined as **List A's Order Number** and **List B's Invoice Number**):

1. **Matched_Records.csv**: Contains records where the Unique Reference exists in **both** Table A and Table B.
2. **Missing_In_B.csv**: Contains records that exist in Table A but are absent from Table B.
3. **Missing_In_A.csv**: Contains records that exist in Table B but are absent from Table A.

NOTE: You can add or deliver anything that you think is needed.

4. Technical Constraints

- **Language:** Any programming language may be used, though **Java** or **C#** is preferred.
- **Data Integrity & Validation:** You must implement robust data validation. Ensure that all fields are checked for correctness (e.g., date formats and numeric ranges) before processing.
- **Error Handling:** The application should handle exceptions gracefully.
- **Performance & Efficiency:** The solution must be optimized for high-volume data. Assume the input files contain millions of rows; your implementation should manage memory efficiently.
- **Clean Code:** Follow industry best practices for code quality. This includes using meaningful naming conventions, modular design, and clear documentation